Information Security

Lec-1

information = Randomness = lack of knowledge

↓

Independence

||

freedom

non random = determinism

Information theoretic security & Computational security

- information can not be determined by ANY COMPUTER.

eg $f(x) = y$ x & func^n ⟶ info^m

Lec-2 01/01/26

Lack of knowledge ⇒ Randomness ⇒ independence ⇒ information

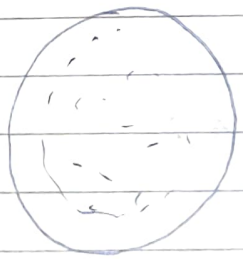
(Randomness: security; independence: free will; ? : Quantum Source)

$A \Rightarrow B$

$\neg B \Rightarrow \neg A$

$B \Rightarrow A$

Classical view of universe



1. Elementary particles
2. Basic forces

→ Even with all knowledge this is still random (Quantum Source)

"The Universe is not Real", Zelenger 2022

"Superdeterminism" → Paul, 2026

≠

Zelinger
↳ universe is not Superdeterminism

Lec-03 05/01/26

Cryptography

- SHA
- RSA
- MD5

→ Cryptographic Hash Function (CHF)

- AES
- RC4
- ECDSA

→ Encryption function / CIPHER + Decryption function

The goal / definition of CHF Concept:

→ Input
→ Output
→ Properties ← one-wayness, collision resistance

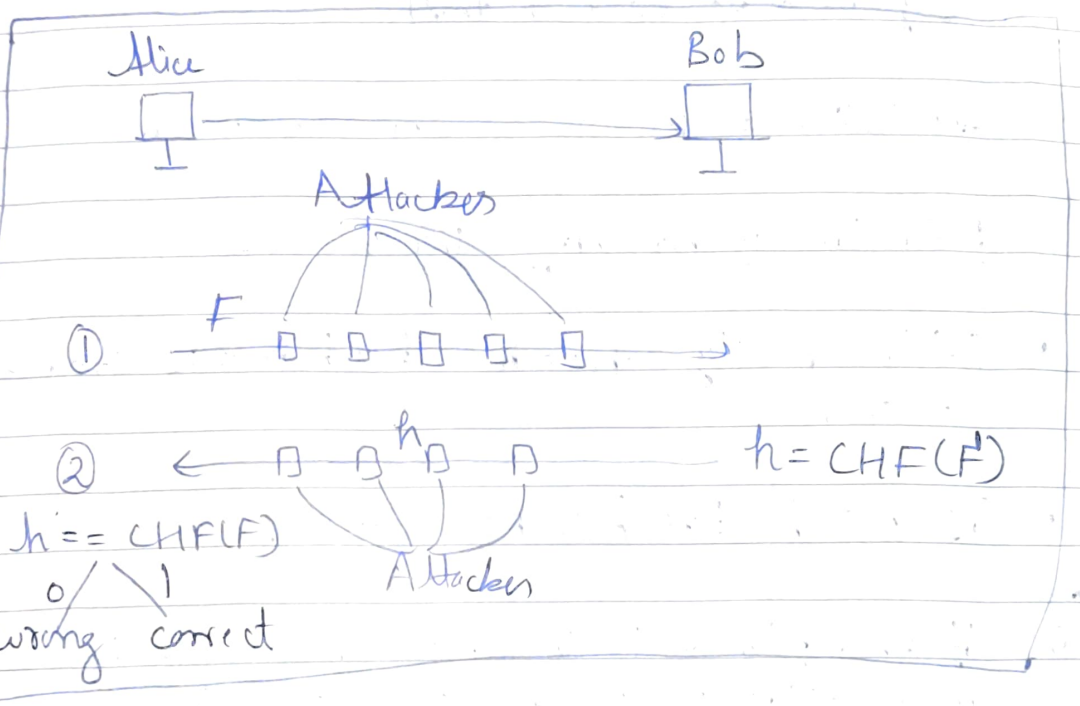
Input: $\{0,1\}^* = \{0,1\} \cup \{0,1\} \cup \{0,1\} \cup \cdots$

Output: ~~$\{0,1\}^*$~~ → $\{0,1\}^a$ → fixed length

- SHA-3 $a = 256$

Meaning of a use-case of a CHF

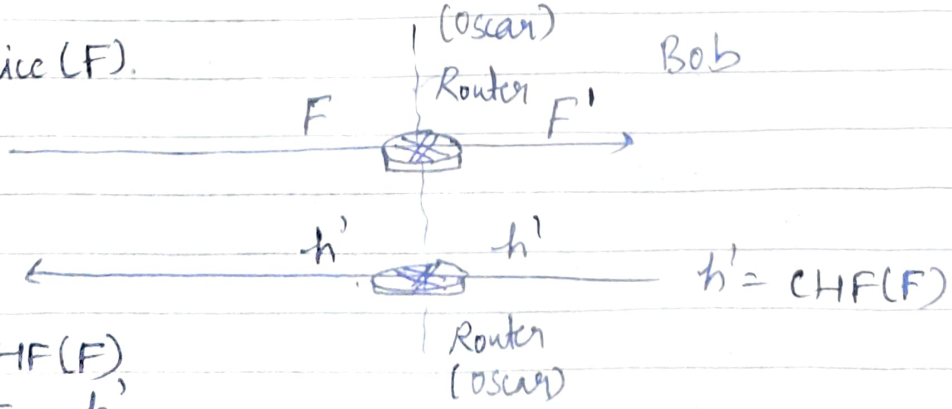
Protocol
- Humans
- Machines
- Communication channel



CHF(F) = F[0..... 255]

Lec-04

Usecase 1



h = CHF(F)
if h == h'
then data authenticated
else data modified

⭐⭐ Q without assumption-2, no matter how much effort we put in protocol, it can be attacked, Prove

Assumption

1. Bob does not modify F
2. Oscar does not modify h'
3. Alice & Bob can not forge their identities (System takes care of this)

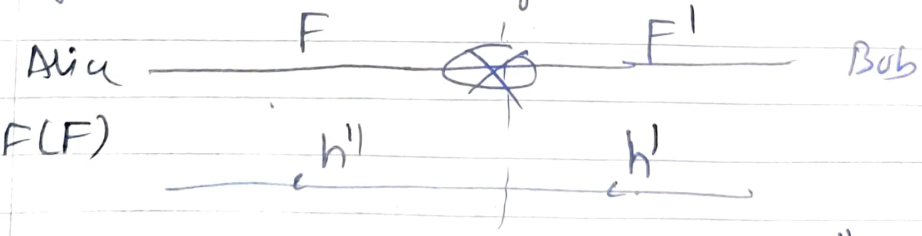
Application = use-case = app = protocol

Definition of breaking Use-case-1

- If in reality $h == h'$ but bob has stored $F' \neq F$
- If $h \neq h'$ but bob has stored $F' = F$

⭐⭐ Q. If Assumption-2 is is removed no matter how much effort we put to design CHF, it can be attacked. Prove

Ans If oscar could modify h' also then



Oscar changes $F \rightarrow F'$ & computes $h'' = CHF(F')$

~~& changes~~ instead of

Now when bob computes $h' = CHF(F')$ let's suppose its not equal to $h = CHF(F)$ but oscar can modify it in b/w & sets $h'' = CHF(F')$

$\Rightarrow$ F changed but data authenticated.

Design Attempt

① CHF: $F \longrightarrow F[0 \ldots 255]$ ✗ → o/p = 1 / 0; → not fixed length; → no security

② CHF: $F \longrightarrow h$ (XOR)

where $h[0] = F[0] \odot F[255]$

$h[1] = F[1] \odot F[254]$

$h[2] = F[2] \odot F[253]$

$\vdots$

$h[255] = F[255] \odot F[0]$

Q.1 if File size is fixed to 255 is this correct?

Ans can be easily broken $h[i] = F[i] \oplus F[n-1-i]$

if attackers changes $F[i] = F[i] \oplus \delta$

then $h[i] = F[i] \oplus \delta \oplus F[n-1-i] \oplus \delta$

$= F[i] \oplus F[n-1-i]$

(same) → collision

Use-Case-2 : Password authentication

(Oscar)

Alice (Password) — Router — Google

1. $h = CHF(Password)$ — h — h' → $h' == h$

Lec-5 08/01/26

Components of any protocol/usecase/application
1) Humans
2) Local Machines
3) Communication channels

Use-case-2 (Password Generation)

Alice ~~Bob~~ Gmail Server



1. Create a password p
2. h := CHF(p)



Assumptions:

1. Gmail Server is honest (not malicious)
2. Alice is honest
3. UserId are publicly known

What is the definition/meaning of "breaking a protocol":

1. Alice creates a password p)
2. Alice sends $h' = H(p)$ to Gmail servers
3. Oscar modifies h' to h'' in the communication channel
4. Gmail receives $h'' \neq h'$ & stores it
   Table[Alice] = h''

Now oscar can send h'' & access Alices gmail account.

M T W T F S S
Page No
Date
YOUVA

Another protocol to break

1. Oscar captures h
2. Oscar brute forces h to get p' s.t $CHF(p') = h$

Prevention / Mitigation

1. Make $CHF(\cdot)$ one way and collision resistant (Attack-2 is prevented)

Lec-6 12/01/26 (Not in Syllabus)

DSL353 Attendance System

Step-1 : Instructor clicks a photograph

Step-2 : Instructor shares the picture with the student.

Step-3 : Instructor marks attendance in the google sheet & share with the student.

Definition of attack
A student not present is marked present.
A student which is present is marked absent.

Assumptions :

1. Instructor is honest photographer
2. The server/database storage is honest and secure hackproof.
3. No student has a camera in the class.

Security

Institute attendance system

1. A student uses a academics controlled card-reader and id card (no processor) to send attendance data to academic server.

2. Attendance Academic server stores attendance data.

3. Academic shares attendance data with the student.

(Real Syllabus)

13/01/26 Lec-7

Usecase-1 (Password generation)

Alice | Server

1. generate Password

2. password, username → 3. T[username] = password

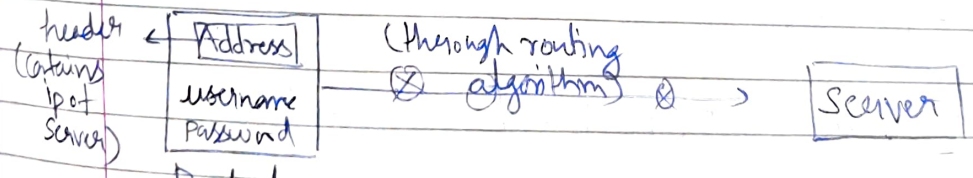
Usecase-2 (Password authentication)

Alice (username, Password)

1. username, password → 2. if T[username] == password then "verified" else "Invalid"

Assumption:

1. Alice & server are honest.



Attack usecase to break usecase-1

1. Oscar captures data = {username = Alice, password = Alice-password} & stops data from reaching the server.

2. Oscar sends to server data = {Alice, oscar-password}

Lec-8 15/01/26

Usecase: 15-01-2026

(San Fransisco) Alice (K) — (Doha) Oscar — (Delhi) Bob (K)



Source Addr	Dest Addr	Data

Oscar can Capture

$E(K, x) = y$ ⟶ y ⟶ $x = D(K, y)$

K → secret information
x → message

Secret encryption Algorithm

Symmetric key encryption
↓
Alice & Bob has same key

$E(K, x)$
$D(K, E(K, x)) = x$

* What security properties $E(\cdot,\cdot)$ & $D(\cdot,\cdot)$ would have so that usecase 15-01-2026 is secure?

Ans
1. Correctness: $D(K, E(K, x)) = x$
2. Confidentiality: without key K, recovering x from y must be computationally infeasible
3. Randomized Encryption: Same plaintext must not always produce same ciphertext
4. Key must be sufficiently large & random
5.

$a \hat{} a = 0$

$0 \hat{} a = a$

M T W T F S S
Page No.:
Date:
YOUVA

Break-1 usecase 15-01-2026

1. Oscar captures y
2. Oscar has an algorithm, say A
   such that $A(y) \longrightarrow x'$
   such that $E(K, x') = y$

$E(K, x) = x^2 = y$	$E(K, x) = K \oplus x = y$
$D(K, y) = \sqrt{y}$	$D(K, y) = y \oplus K = x$
	$\hookrightarrow (K \oplus x) \oplus K = x$

Break-2 usecase 15-01-2026

1. Oscar ~~captures~~ sends x to Alice & asks her to encrypt x and send to him y.
2. Oscar captures y' (evening me)
3. Oscar determines (K)

Home-Assignment

Implement Standard encryption/decryption algorithm (AES-128) find an open-source implementation & try to encrypt your files.

Lec-9 23/01/26

Alice (secret key K) | Bob (Secret key K)

1. C := E(K, m)



m = D(K, C)

* → Except secret key K all are publicly known
→ m = mail / message

1. Without a shared secret K Alice CANNOT send confidential message to Bob.

2. Without meeting in-person a secret-key can not be shared

Is there a fallacy?
YES

Encryption key = K
Decryption key = K

→ Decryption key is more important than the encryption key (if decryption key is leaked, msg will be leaked.)

→ Secret keys need not to be identical (but should be related).

$$K' = f(K)$$

↳ also known

PKC → only ensure confidentiality not authenticity

A → B
1GB

RT very small

M T W T F S S
Page No:
Date:
YOUVA

Suppose:
→ Both Encryption key (K) and Decryption Key (K') are with Bob

→ It is hard to compute K' from K
↳ Bob can publish the encryption key (K)
⇒ K is no more a secret key
⇒ Alice can encrypt the message using publicly known encryption key

Reason: Since it is computationally hard to compute K' from K we can share K publicly.

Alice (encrypted key K) — Bob (encrypted key K, decrypted key K')

C := E(K, m)

C →

m = D(K', C)

→ Encrypted key := Public key
Decryption key := Private Key

eg. 1. MD5 (CHF) → function
2. DES (SE (Symmetric encryption)) } definition
3. RSA (Public key encryption) }

||PKE|| ≃ 10^5 · ||SE|| (but sir said 10^4)

↳ Time complexity

4. DL Key established

Q How Alice can send 1 GB of data to Bob using PKE with less computational time (SE is already ~~encrypted~~ implemented).

Sol^n Alice can send the symmetric encryption key to Bob by encrypting it with the public key
⇒ Now Alice can encrypt message using Symmetric key & send that to bob.

Steps:

1. encrypt symmetric key using public key & Alice sends it to bob.
2. Now Alice encrypt message using SE & sends it to Bob.
3. Bob decrypts the SE key using private key & then decrypts the message using the SE key.

Alice (x) — Oscar — Bob (y)

① $x' = f(x)$

② x' → ③ $y' = f(y)$

④ y'

$[x, x', y']$ $[x', y', y]$

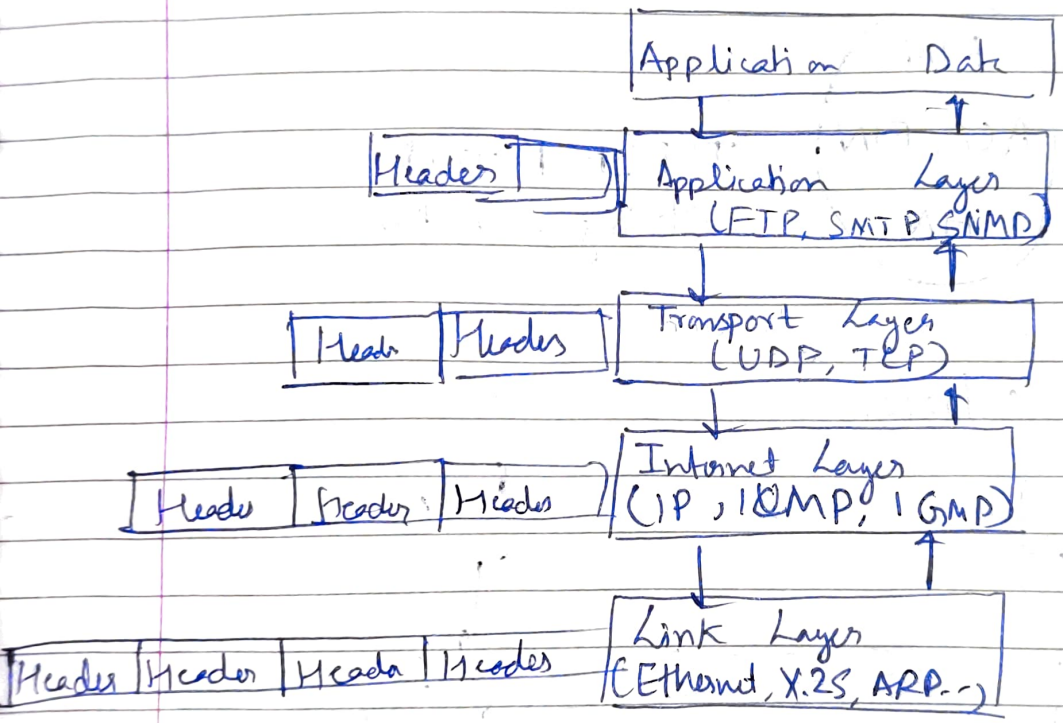
$G_1(x, x', y') = z$ $G_1'(x', y', y) = z$

Oscar can't know z because he doesn't know x & y

Lec-10 3/02/26

1) Encryption
   - Symmetric
   - Public Key

2) Hash function



→ serial no. of NIC card

Q Why do we need MAC Address & IP Address (MAC is unique for each device).
- MAC address identifies device inside local network
- IP used for routing
- MAC used for frame delivery

→ Switches do not understand IP, Local delivery works using Ethernet & Ethernet use MAC addresses.
(MAC only works inside one Local network)

SYN
ACK, SYN
SYN

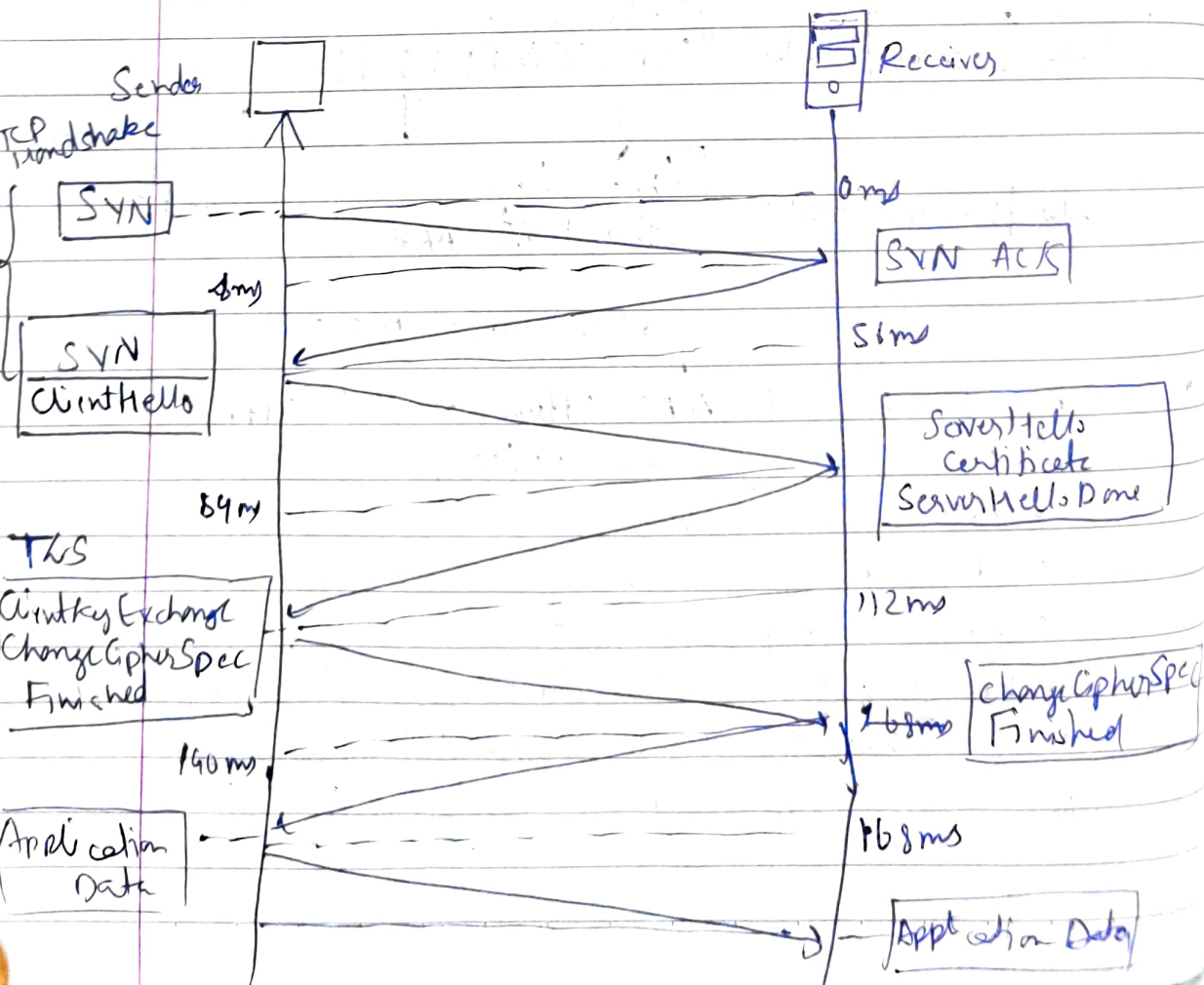
TCP/IP Protocol
3-Way Handshake

Application Layer Protocol = http, https, VPN, DNS, DNS sec

FIN
ACK, FIN
FIN

TCP/IP Protocol
Closing connection

Lec-11 04/02/21



Cryptographic algorithms

1) Symmetric encryption system (Eg. DES, AES)
2) Public Key Encryption system (Eg. RSA, ECC)
X 3) Hash function (Eg. SHA-1/2/3, MD5, MD4)
4) MAC = Message authentication code (Eg. HMAC)

we will not use hash func^n in this usecase

ClientHello ⟶ client sends all the cryptographic algo available in its browser to server

ServerHello
certificate
ServerHello Done
⟶ Server gets the list, both client & server accepts ~~one~~ one algo & server sends algo details along with public key.

ClientKeyExchange
ChangeCipherSpec
Finished
⟶ client creates a random no. (Symmetric key) & encrypt it using public key & sends back to server

ChangeCipherSpec
Finished
⟶ Server receives the encrypted msg & decrypt it using its private key to get the random no.

⇒ Now both client & server shares the random no (only valid for this session)

Lec-12 05/02/26

Q. How client verifies that it is sending request to the verified & real servers?

Ans: Certificate

Public Key signature scheme (PKS scheme)

Two functions:
1. Sign()
2. verify()

Sign(P_r, m) ⟶ σ (cryptographic signature)
- P_r: Private Key
- m: message
- σ: 300 bits

Verify(P_u, σ, m) ⟶ b $\in \{0, 1\}$ (bit)
- P_u: Public Key
- σ: Signature
- m: msg

if b = 1 then the tuple (P_u, σ, m) are consistent ⇒ private key is also correct.

Home Assignment: Analyze verify algorithm {Significance of b = 0/1}

(Simple) Usecase:

→ Not a good identity verification protocol,
1. P_u is not verified.
2. Oscar can catch σ, m & then pretend to be Alice.
   → Use random challenge

V(Pu, Sign(Pr, M), r)

(Simple) Usecase :

→ if challenge is not random then this is not safe (random means → always generate fresh random no.)

Alice (Pr) {Pu} Bob

← r (random challenge)

σ := Sign(Pr, r)

σ →

b := verify(Pu, σ, r)

if b=1 this is alice else forgery

(σ doesn't changes)

→ Let's suppose random generator is bad ⇒ oscar can get the σ once & then he can pretend to be Alice by sending σ again & again.

Q. How to check public key is correct (verified)?

→ Do physical verification once instead of verifying digitally everytime. (Certificate)

Lec-13 06/02/26

* Public Key Identity information
  - IP Address
  - MAC Address
  - Port Address

} Identity documents (used to prove the identity)

→ Pu key & Pr key can't be generated by certifying agency. Have to be generated by Applicant only.

M T W T F S S
Page No.:
Date:
YOUVA

Identity document generation protocol

CERTIFICATE

Applicant — **CA** (Certifying agency)

1. Generates Pr/Pu

$<DOC> = Pu, Name, Dob, Address, Father's Name$

Walk (not sending online) because someone could modify it.

$CERT_A = (<DOC>, Sign(<DOC>, Pr_{CA}))$ — Pr_{CA}: Private key of CA

Stores <DOC>, $CERT_A$ ← $CERT_A$ (walk not online) {can be online also}

Alice (Pr_A, Pu_A) — **Bob**

Identity document verification

r →

$\sigma = Sign(Pr_A, r)$

σ →

→ but before this bob has to verify the public key (Pu_A)

$b := Verify(\sigma, r, Pu_A)$

Identity document Verification protocol

Bob

$b = Verify(<DOC>, Pu_{CA}, CERT_A)$

random challenge is not required for identity document verification.

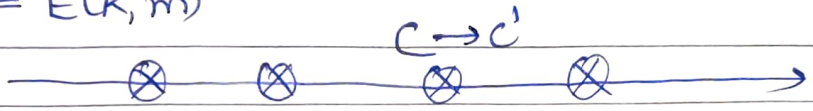
Lec-14 10/02/26

1) Public Key Signature Algorithm (PKS) → certificate generation & verification
2) Symmetric Key Encryption / Decryption (SE)
3) Message Authentication Code (MAC)
4) Deffie Hellman Key Agreement protocal (DH)

Message Authentication Code (MAC)

Alice(K) {E(·), D(·)} Bob(K) {E(·), D(·)}

1. C = E(K, m)



m = D(K, C)
m' = D(K, C')

Q What security property E(·)/D(·) achieves in the above protocal?

Ans Confidentiality (Now routers in between can't read the data)

Data Authenticity ✗ (someone can change the actual data → receiver will not receive the authentic data)

MAC

Alice(K)

1. C = E(K, m)
2. tag = MAC(K, m)

tag = H(m)
what if we use Hash fⁿ (EXAM★★)

if C → C'
suppose tag → tag'
but oscar can't send exact tag' because he doesn't have K

Bob(K)

C, tag

3. m = D(K, C)
4. tag' = MAC(K, m)
5. tag == tag'

Discard Accept

Diagram: (Internet)

Diffe Hellman Protocal

Alice(x) {α, p}

Bobly) {α, p}

1. $\alpha^x \bmod p$

$\alpha^x \bmod p$ →

← $\alpha^y \bmod p$

2. $\alpha^y \bmod p$

3. $(\alpha^y)^x \bmod p = K$
$= \alpha^{xy} \bmod p$

4. $(\alpha^x)^y \bmod p = K$
$= \alpha^{xy} \bmod p$

Alice (x)

Oscar

Bobly)

1. $4x$

$4x$ →

2. $3y$

← $3y$

3. $\frac{3y}{3} \rightarrow y$

4. xy

5. $\frac{4x}{4} \rightarrow x$

5. xy

Assumption

1. Given a, α, p, it is hard to compute x
2. Given b, α, p it is hard to compute y.

Lec-15 11/02/26

Q. Design a protocol where Alice identifies herself to Bob.

Ans

Lec-16 12/02/26

Client Server

Searchword

Searchword → Results ← f(Searchword)

Results ←

https = http + SSL (Secure socket layer)

1. Packet: Searchword = "SP"
2. Headers of the packet
3. Communication steps in sequence.

Client HTTP Server

IP Packet

A := [...][...][SYN][<Empty> Data] ——A——→

B := [..][...][SYN ACK][<Empty>]

←——B——

C : [...][...][ACK][<Empty>]

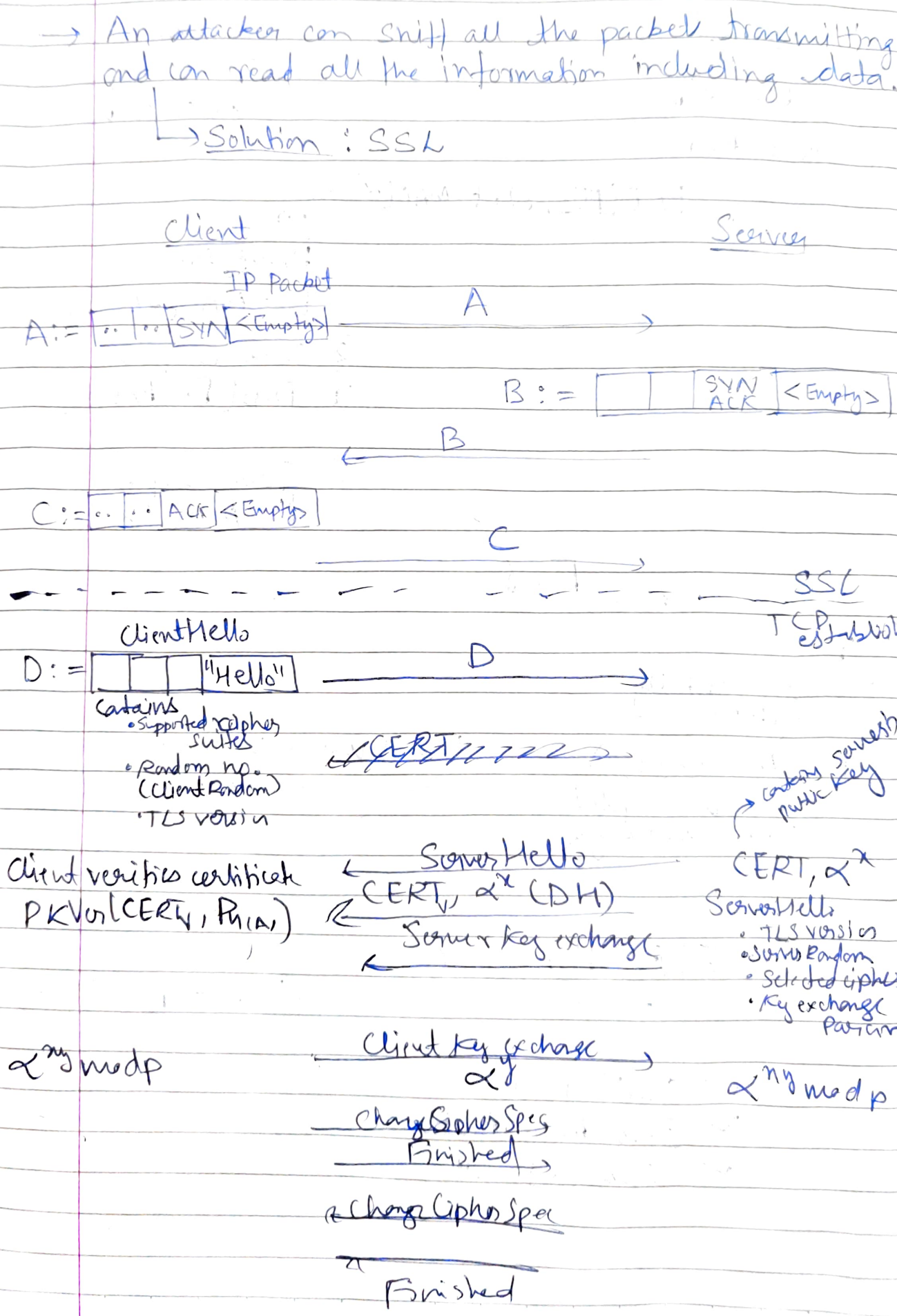
——C——→

[...][..][..]["SP"] ——→ D = f("SP")

←—— [...][...][..][D]

Apps:
→ does not use PKS
→ client doesn't have any private key or public key. (reason → requires money to get CA)

→ An attacker can sniff all the packet transmitting and can read all the information including data.
→ Solution : SSL



Client

$K = \alpha^{xy}$

generates random x

Server

$K = \alpha^{xy}$

$PKE(PU_s, r), E = MAC(\alpha^{xy}, 0) \rightarrow$

① Verify MAC
$E == MAC(\alpha^{xy}, 0)$

② Server computes
$r = PKD(PR_s, PKE(PU_s, r))$

③ Server computes
$F = MAC(K, 0)$

$\leftarrow F, r$

① $r == r$
→ Server succesfully decrypted r.

② $F == MAC(\alpha^{xy}, 0)$
→ Server also computed same shared key K

Now mutual confirmation both share same K

$G_1 = E(K, "SP") \quad G_1 \rightarrow$

$"SP" = D(\alpha^{xy}, G_1)$

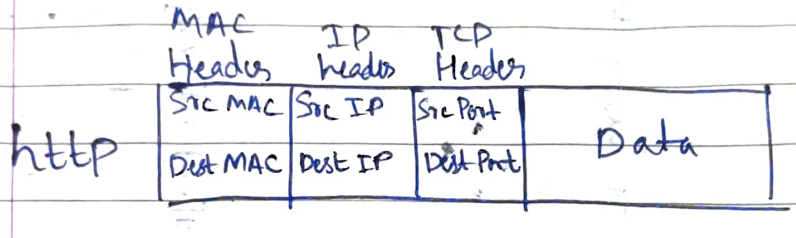
- ✓ Confidentiality
- ✓ Server Authentication
- ✓ Key confirmation

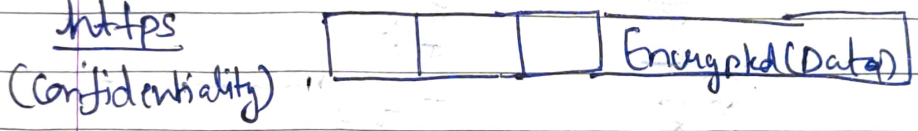
M T W T F S S
Page No.:
Date:
YOUVA

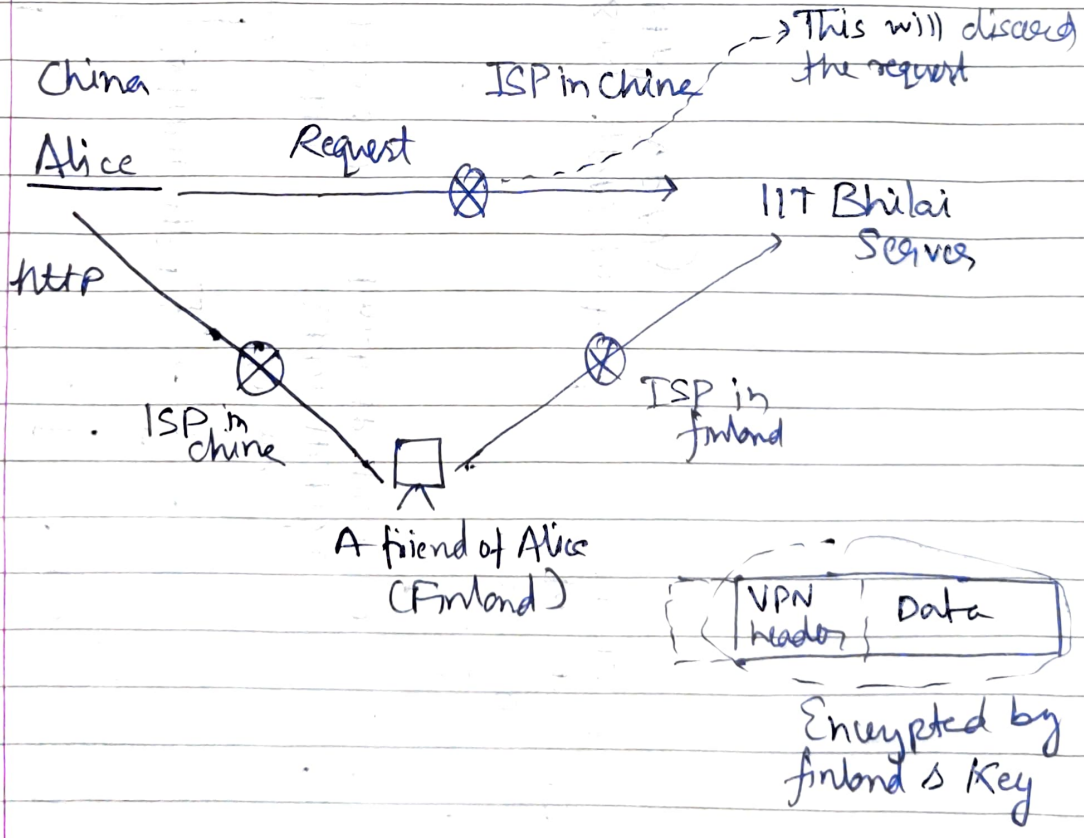
Lec-17 14/02/26

Internet usecase-2 : VPN protocal

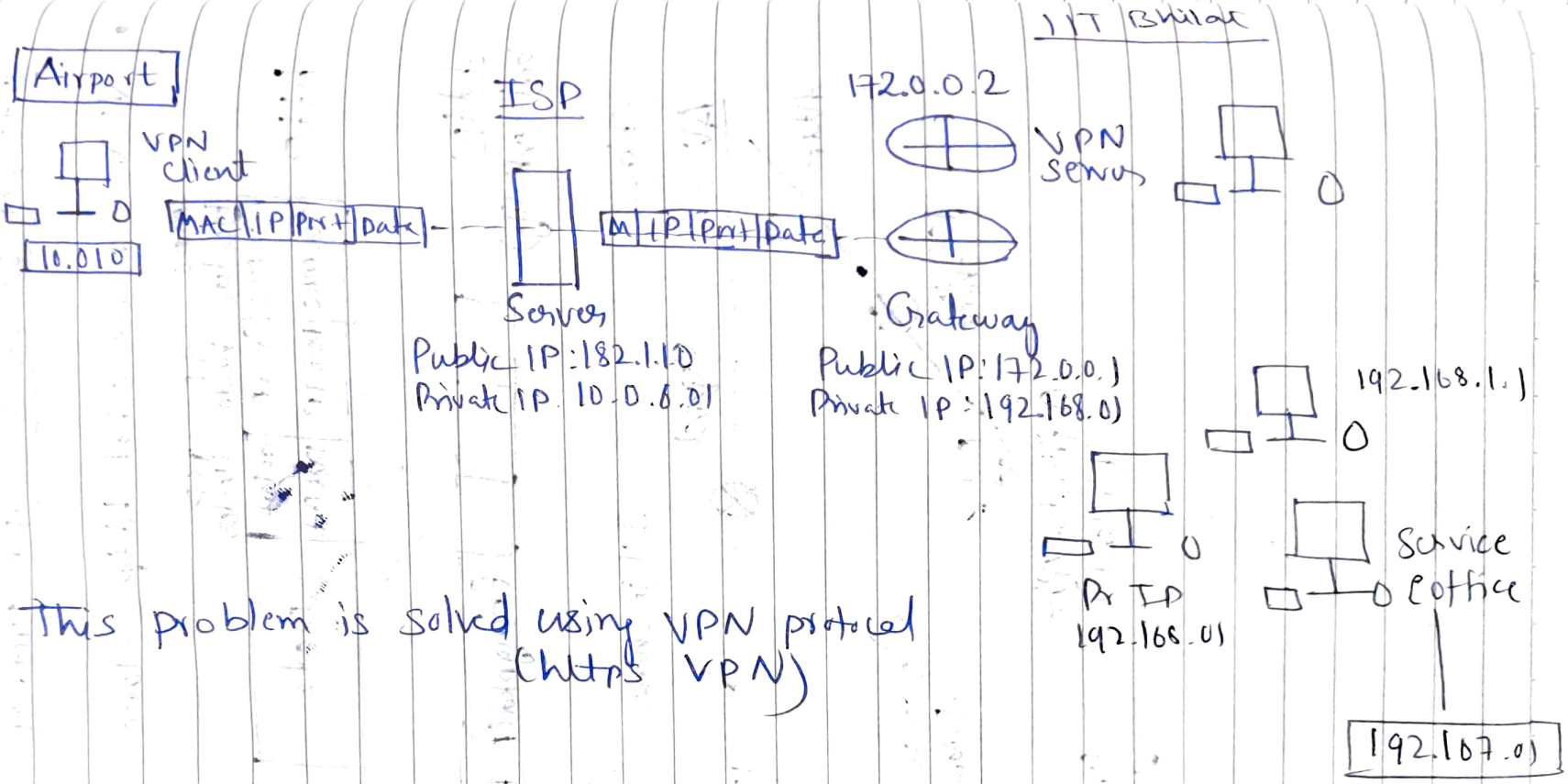
- VPN achieves what http "alone" did not achieve



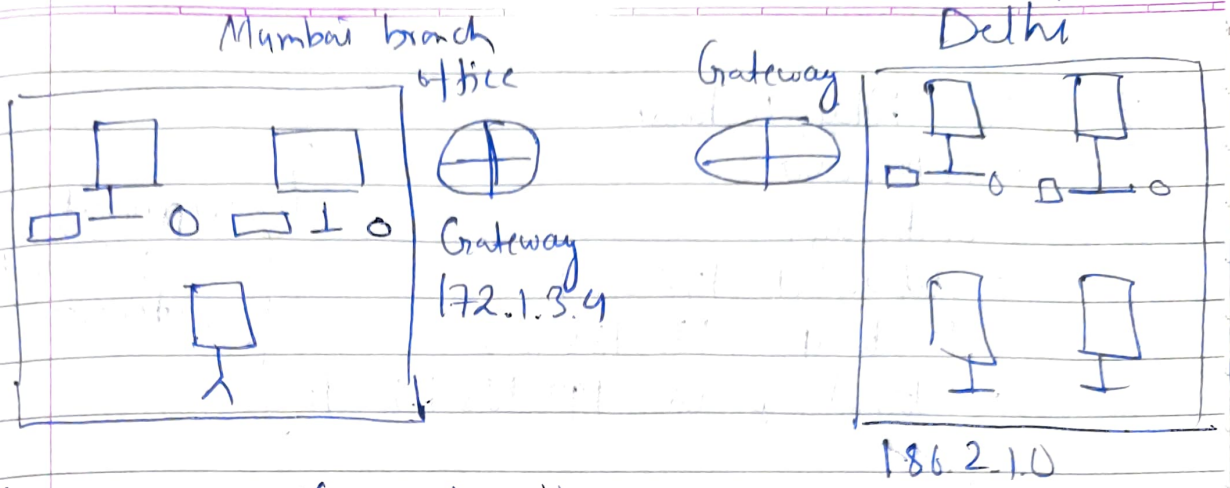




Lec-18 16/02/26



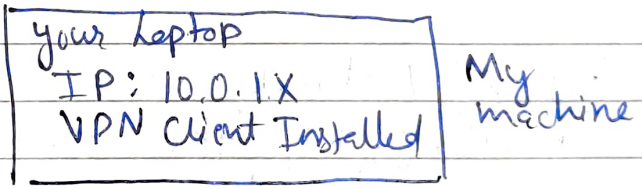
This problem is solved using VPN protocol (https VPN)



1) https VPN (SSL VPN)
2) IPsec VPN

Explanation: (HTTPS VPN)

① Airport
- airport network gives you a private IP Addr (eg 10.0.1.X)
- This is not globally routable
- you are inside a local LAN at airport



② Middle (ISP)
- public internet
- Airport LAN → goes through ISP → reaches public internet
- ISP routers fwd traffic to dest public IPs

So when we connect to VPN server, packet travels
Airport LAN → ISP → Public Internet → IIT Bhilai gateway

③ IIT Bhilai Network

- VPN servers
- Gateway
  - Public IP: 172.0.0.1
  - Private IP: 192.168.0.X
  - Internal systems: 192.168.1.1, etc

} represent IIT Bhilai network

Gateway:
Public IP (172.0.0.1) → visible to internet
Private IP (192.168.0.X) → inside IIT LAN

gateway does:
- Accept VPN connections
- Authenticate users
- Decrypt traffic
- Fwd traffic to internal servers

Internal Network
e.g. 192.168.1.1, Office, Service servers
these are private network within inside IIT.
(not accessible from internet directly)

Problem:
- I am at airport, IIT Bh server → 192.168.1.1
- I can't access it directly
- Private IPs are not routable over internet.

How VPN solves it

① VPN client initiates connection

my laptop:
Src IP: 10.0.1.X
Dest IP: 172.0.0.1 (IIT Gateway Public IP)

→ I connect to IIT VPN server

② Secure Tunnel creation

using SSL/TLS
HTTPS VPN
Encrypted tunnel

my laptop ═══ secure tunnel ═══ IIT Gateway

③ Encapsulation

when I try to access 192.168.1.1
VPN does:

Original packet
Src: 10.0.1.X
dst: 192.168.1.1

Can't travel over internet

VPN encapsulates it →

Outer packet:
Src: 10.0.1.X
dst: 172.0.0.1
Encrypted payload:
Src: 10.0.1.X
Dst: 192.168.1.1

outer packet travels through ISP

④ Decryption at IIT Gateway

At gateway: - outer layer removed
- Decrypted
- original packet restored

Now gateway finds internally
Src: 10.0.1.X (virtual VPN id), Dst: 192.168.1.1

IPsec VPN = Internet Protocol Security

→ used for site-to-site VPN

→ works at Network Layer (encrypts entire Packet)

Step-1 IKE Phase-1

→ To create a secure channel b/w gates

Procedure:

(i) Gateway A → Gateway B: (support AES-256, SHA-256, DH Grp14, Pre-shared key)

Now Gateway B checks compatibility → reply yes

(ii) DH key exchange (both agree on this)

- generate a shared secret
- without sending secret over network.

(iii) Auth

they verify identity using

- Pre-shared key or digital certificates

Result: Secure encrypted channel created (ISAKMP SA)

Step-2 IKE Phase-2 (Build Data tunnel)

→ how actual traffic will be encrypted

→ they agree on

- ESP protocol
- Encryption algo (AES, 3DES)
- Integrity algo (SHA)
- Lifetime of tunnel

Result: IPSec SA, defines how data packets will be encrypted.

Step-3 : Data Transfer

Mumbai PC
Src: 192.168.10.5
Dst: 182.1.0.20 → ⊕

Mumbai gateway checks is dest inside VPN subnet? → Now it:
- encrypts entire packet
- wraps it inside new IP header
- sends to delhi gateway public IP

IP Header TCP Header Data	→ IPSec tunnel mode →	Outer IP Header (Public IPs) ESP Header Encrypted Original IP header TCP Header Data ESP Trailer ESP Auth
Before		After

Internet only sees
Public IP → Public IP

Step-4 Decryption at other side

Delhi gateway → (i) recognizes ESP packet
(ii) Uses security association
(iii) Decrypts payload
(iv) restores original pkt
(v) forwards internally

Step-5: Rekeying

each SA has a lifetime → time based (1 hr)
→ data based (1 GB)

when expires:
- phase 2 runs again
- new keys generated.